

NP-Complejidad

Comentario: Ningún ejercicio reviste mayor dificultad.

Ejercicio 1. Responder breve y claramente:

a. ¿Por qué en la complejidad espacial se utilizan MT con una cinta de entrada de sólo lectura?

En la complejidad espacial, separamos la cinta de entrada (solo lectura) de las cintas de trabajo (lectura/escritura) para poder medir el espacio sub-lineal (menor a n).

Si la cinta de entrada fuera de lectura/escritura, cualquier algoritmo ocuparía automáticamente al menos $O(n)$ de espacio solo por el hecho de tener la entrada allí. Al hacerla de solo lectura, podemos estudiar problemas que requieren muy poco espacio, como $O(\log(n))$.

El objetivo es medir cuánta memoria adicional (cintas de trabajo) necesita el algoritmo para resolver el problema, sin que se vea afectado por el tamaño de los datos que le damos para leer.

Además, si pudiéramos escribir en la cinta de entrada, la máquina podría usar ese espacio para guardar cálculos intermedios. Esto "escondería" el uso real de memoria que estamos tratando de analizar.

b. ¿Por qué si una MT tarda tiempo $\text{poly}(n)$ entonces ocupa espacio $\text{poly}(n)$, y si ocupa espacio $\text{poly}(n)$ puede llegar a tardar tiempo $\text{exp}(n)$?

- ¿Por qué si una MT tarda tiempo $\text{poly}(n)$ entonces ocupa espacio $\text{poly}(n)$?

En cada paso de ejecución, el cabezal de una Máquina de Turing solo puede moverse una posición o escribir en una celda nueva. Si la máquina funciona durante un tiempo polinomial $p(n)$, no hay forma física de que llegue a tocar más de $p(n)$ casilleros de la cinta. Por lo tanto, el espacio utilizado siempre está "atrapado" por la cantidad de pasos ejecutados, cumpliéndose que $\text{Espacio} \leq \text{Tiempo}$. Si el tiempo es un polinomio, el espacio no tiene otra opción que serlo también.

- ¿Por qué si una MT ocupa espacio $\text{poly}(n)$ puede llegar a tardar tiempo $\text{exp}(n)$?

Aunque el espacio sea limitado, podés combinar los símbolos de la cinta de muchísimas maneras diferentes reutilizando el mismo espacio. Con un espacio polinomial $S(n)$ y un alfabeto de k símbolos, existen $k^{S(n)}$ contenidos de cinta distintos, lo que representa una cantidad exponencial de **configuraciones** posibles. Para no entrar en un bucle infinito, una máquina determinista puede recorrer todas estas combinaciones antes de detenerse, tardando un tiempo que crece exponencialmente respecto al tamaño del espacio disponible.

c. ¿Por qué los lenguajes de la clase LOGSPACE son tratables?

Contexto:

- **Concepto de tratable:** un problema se define como "tratable" si pertenece a la clase P (es decir, se resuelve en tiempo polinomial).

¿Por qué LOGSPACE $\subseteq$ P?

Como dijimos en el ejercicio anterior, el tiempo máximo que puede tardar una MT está acotado por la cantidad de configuraciones distintas que puede tener.

Si una máquina usa $S(n) = O(\log(n))$ de espacio, la cantidad de configuraciones posibles es aproximadamente $k^{O(\log(n))}$, donde k es una constante (relacionada al alfabeto y los estados).

Por propiedades matemáticas, $k^{O(\log(n))}$ es equivalente a $n^{\log k}$, lo cual es una función polinomial (n^c).

Como cualquier algoritmo que use espacio logarítmico está obligado a terminar en un número de pasos polinomial para no repetir configuraciones, automáticamente cae dentro de la clase P y, por lo tanto, es tratable.

Ejercicio 2. Describir la idea general de una MT M que decida el lenguaje $L = \{a^n b^n \mid n \geq 1\}$ en espacio logarítmico.

Ayuda: basarse en el ejemplo mostrado en clase.

La siguiente MT decide L en espacio $O(\log n)$:

Idea general:

- 1) Verifica que el primer símbolo sea una 'a'. CC rechaza
- 2) Se va desplazando a derecha contando la cantidad 'a's e incrementando un contador en la cinta de trabajo 1.
(Si le viene un símbolo distinto de 'a' o 'b' mientras se desplaza, rechaza)
- 3) Al encontrar la primera 'b', repite contando la cantidad de 'b's e incrementando un contador en la cinta de trabajo 2.
(Si le viene un símbolo distinto de 'b' mientras se desplaza, rechaza).
- 4) Al encontrar Blanco en la cinta de entrada, compara los contadores.
 - Si son iguales, acepta.
 - Si son distintos, rechaza.

Esto es $O(\log n)$ porque:

- La cinta de entrada es de solo lectura.
- Los contadores ocupan espacio $\log(n)$ con respecto a la entrada, asumiendo cualquier sistema de representación numérica distinto del unario.

Ejercicio 3. Describir la idea general de una MT M que decida el lenguaje SAT en espacio polinomial.

Ayuda: la generación y la evaluación de una asignación de valores de verdad se pueden efectuar en tiempo polinomial.

Idea General de la MT:

Para una fórmula booleana ϕ con n variables:

- 1) La máquina utiliza la cinta de trabajo para escribir UNA posible asignación de valores de verdad (por ejemplo, empezar con todas las variables en "falso": 0000)
- 2) Se evalúa si la fórmula ϕ es verdadera con esa asignación.
Como la evaluación se puede hacer en tiempo polinomial, sabemos que ocupará un espacio polinomial en la cinta.
- 3) Si la fórmula es verdadera, acepta.
Si no es verdadera, la máquina reutiliza el espacio de la cinta de trabajo para generar la siguiente asignación (por ejemplo, sumando 1 en binario para obtener 0001).
- 4) Si la máquina recorre todas las 2^n asignaciones posibles y ninguna hace que la fórmula sea verdadera, rechaza.

Por qué SAT $\in$ PSPACE?

- Guardar los valores de n variables solo requiere $O(n)$ celdas, lo cual es polinomial respecto a la entrada.
- La clave es que no necesitamos guardar todas las asignaciones a la vez. Al borrar y sobrescribir la asignación anterior, el espacio total utilizado en cualquier momento de la ejecución nunca supera un límite polinomial. Aunque este algoritmo tarda un TIEMPO exponencial en revisar todas las combinaciones, el espacio se mantiene pequeño (polinomial) gracias a la reutilización constante de las mismas celdas de la cinta.

Ejercicio 4. Probar que $NP \subseteq PSPACE$.

Ayuda: Si L pertenece a NP , entonces existe una MT $M1$ capaz de verificar en tiempo $poly(|w|)$ si una cadena w pertenece a L , con la ayuda de un certificado x de tamaño $poly(|w|)$. De esta manera, existe también una MT $M2$ que decide L en espacio $poly(|w|)$, sin la ayuda de ningún certificado.

La lógica es similar a la del ejercicio anterior, de $SAT \subseteq PSPACE$:)

Si L pertenece a NP , por definición existe una $M1$ que la verifica en tiempo $poly(|w|)$, y un certificado de espacio $poly(|n|)$!

Entonces, idea general de $M2$:

- 1) Genera el primer posible certificado en orden canonico.
- 2) Ejecuta $M1$ sobre ese certificado.
- 3) Si $M1$ acepta, $M2$ acepta.
Si $M1$ rechaza, $M2$ genera el siguiente certificado, reutilizando el espacio del primero. Vuelve al paso 2.
- 4) Si termina de probar todos los posibles certificados y $M1$ no aceptó, $M2$ rechaza.

Por qué $NP \subseteq PSPACE$? similar al de SAT :

- Guardar una cadena x de tamaño polinomial requiere espacio $O(poly(|w|))$
- Como $M1$ corre en tiempo polinomial, el espacio que utiliza para sus cálculos también es $O(poly(|w|))$ (tiempo determina espacio).
- NP puede tardar mucho tiempo (exponencial) probando certificados, pero como puede "reciclar" el espacio para cada intento, no necesita más que un espacio polinomial para decidir el problema.

Ejercicio 5. Supongamos que existe una MT M de tiempo polinomial que, dado un grafo G , devuelve un circuito de Hamilton de G si existe o responde no si no existe. Describir la idea general de una MT M' que, utilizando M , decida en tiempo polinomial si un grafo G tiene un circuito de Hamilton.

Ayuda: basarse en el ejemplo mostrado en clase con FSAT y SAT. +

Es intuitivo que, si tenemos una MT M que busca un circuito de hamilton y, en caso de existir, lo devuelve en tiempo polinomial, entonces claramente podemos hacer una MT M' que decide en tiempo polinomial el mismo problema, ya que es el tiempo de MT M + un pquito.

Idea general de MT M' :

- 1) M' recibe el grafo, y ejecuta a MT M pasándole el grafo.
- 2) La máquina M ejecuta y se toma el trabajo pesado de buscar si existe.
- 3) M' analiza la salida:
 - Si M le devolvió un grafo: acepta
 - Si M le dijo no: rechaza

¿Por qué sigue siendo tiempo polinomial?

- Sabemos que M tarda tiempo polinomial ($\text{poly}(n)$). Como el tiempo limita el espacio (ejercicio 1b), entonces, sabemos que M usa espacio $\text{poly}(n)$.
- M' solo tiene que llamar a M y mirar lo que devolvió (eso es casi instantáneo).
- Por lo tanto, el tiempo total de M' es el tiempo de M más un poquito más, lo cual sigue siendo polinomial.